# N-step bootstrapping
# Eligibility Trace

이상현

# Index

1. n-step TD
2. n-step SARSA
3. n-step Off-policy Learning
4. Off-policy Learning Without Importance Sampling: The n-step Tree Backup Algorithm
5. TD(λ)
6. SARSA(λ)

# n-step TD Prediction

**MC**

Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha(\boldsymbol{G_t} - V(S_t))$$

High variance, zero bias

- $G_t$는 $v_\pi(S_t)$ 의 unbiased estimator

**TD**

Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha(\boldsymbol{R_{t+1}} + \boldsymbol{\gamma V(S_{t+1})} - V(S_t))$$

Low variance, some bias

- TD target은 $v_\pi(S_t)$ 의 biased estimator

둘 사이를 interpolate한다면?? → n-step idea

- Bias-Variance Tradeoff 활용 + backup depth/reward propagation/credit assignment 조절

# n-step TD Prediction

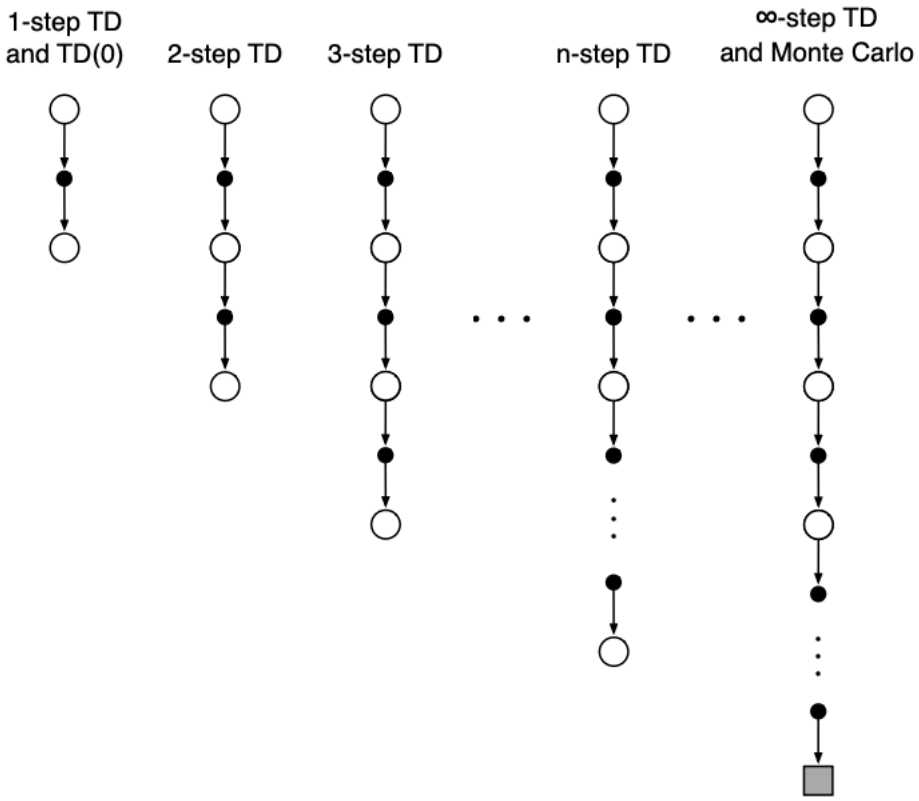
n-step Return

$$
\begin{aligned}
n = 1 \quad (TD) \quad & G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \\
n = 2 \quad & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \\
\vdots \quad & \vdots \\
n = \infty \quad (MC) \quad & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T
\end{aligned}
$$

n-Step TD Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$$

교과서에서는 $G_{t:t+n}$ 표기를 사용합니다.



# n-step SARSA

$$\text{TD Update Rule:} \quad V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

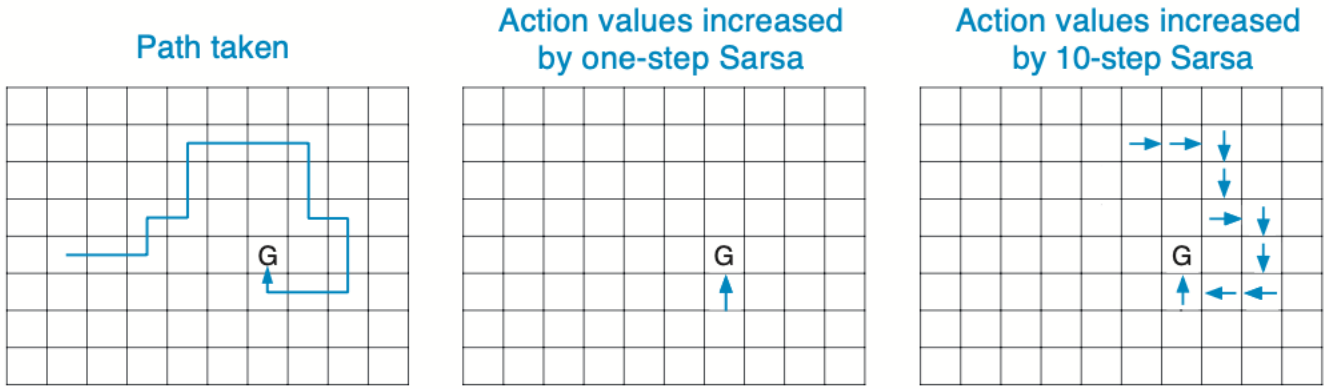
$$\text{SARSA Update Rule:} \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

$$\text{n} - \text{step TD Update Rule:} \quad V(S_t) \leftarrow V(S_t) + \alpha(G_{t:t+n} - V(S_t))$$

$$\text{n} - \text{SARSA Update Rule:} \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(q_t^{(n)} - Q(S_t, A_t)\right)$$

$$q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

# n-step SARSA




기존의 one-step SARSA는 Reward가 한 step 앞의 state-action으로만 전파됨

n-step SARSA의 경우 n-step 전의 state까지 reward가 전파됨.

**→ 한 에피소드 기준으로 n-step SARSA는 리턴을 더 멀리 전파할 수 있다.**

# n-step Expected SARSA

**Expected SARSA**

$$\boldsymbol{R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a)} - Q(S_t, A_t)$$

**n-step Expected SARSA**

$$\boldsymbol{R_{t+1} + \gamma R_{t+2} + \cdots \gamma^{n-1} R_{t+n} + \gamma^n \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a)} - Q(S_t, A_t)$$

SARSA는 현재 정책 $\pi$에서 액션을 샘플링

Expected SARSA는 현재 정책 $\pi$의 분포 자체를 사용

# n-step Off-policy Learning

**Importance Sampling**

$$\rho_{t:t+n} = \prod_{k=t}^{\mathrm{mint}\,(t+n-1,T-1)} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

**n-step TD의 Importance Sampling**

$$V(S_t) \leftarrow V(S_t) + \alpha\rho_{t:t+n}(G_{t:t+n} - V(S_t))$$

**n-step SARSA의 Importance Sampling**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\rho_{t+1:t+n}\big(G_{t:t+n} - Q(S_t, A_t)\big)$$

* n-step Expected SARSA 의 경우 마지막 액션을 샘플링하지 않기 때문에 $\rho_{t+1:t+n-1}$을 사용한다.

# Per-decision Methods w/ Control Variates

**Recursive expression of n-step return**

$$G_{t:h} = R_{t+1} + \gamma G_{t+1:h}$$

Off-policy의 경우 $\rho_t$가 곱해짐. 이때, $\rho_t = 0$ 이면 update target도 0이 되고

$$V(S_t) \leftarrow V(S_t) + \alpha\big(0 - V(S_t)\big)$$

$V(S_t)$ 를 0쪽으로 업데이트함.

$\rho_t = 0$의 원래 의도는, 타겟 정책에서 샘플링될 수 없는 행동정책 샘플을 **무시**하는것.
타겟 정책에서 샘플링될 일 없는 정책이 가치함수를 떨어뜨리는건 뭔가 이상함

# Per-decision Methods w/ Control Variates

**n-step return with control variate**

$$G_{t:h} = \rho_t(R_{t+1} + \gamma G_{t+1:h}) + (\mathbf{1} - \boldsymbol{\rho_t})\boldsymbol{V_{h-1}}(\boldsymbol{S_t})$$

On-policy 일 경우 $G_{t:h} = R_{t+1} + \gamma G_{t+1:h}$

타겟 정책이 이 액션을 절대 고르지 않으면 $G_{t:h} = V_{h-1}(S_t)$

**Importance sampling을 사용해서 sample path를 타겟 정책에 맞게 보정하되, target policy와 맞지 않는만큼 현재 Expectation으로 가중하는 구조.**

# Off-policy Learning w/o Importance Sampling

**Off-policy multi-step learning을 importance sampling 없이 할 수 있을까?**

One-step에서는 가능했다. (Q-Learning, Expected Sarsa)

n-step으로 확장이 가능한가? → **n-step Tree Backup Algorithm**

Tree backup은 sample path에서 **실제로 선택되지 않은 action들도 고려**한다.

선택되지 않은 action들은 sample reward가 없는데 어케 고려함? → $\boldsymbol{Q(s,a)}$를 **Bootstrap**

# Off-policy Learning w/o Importance Sampling

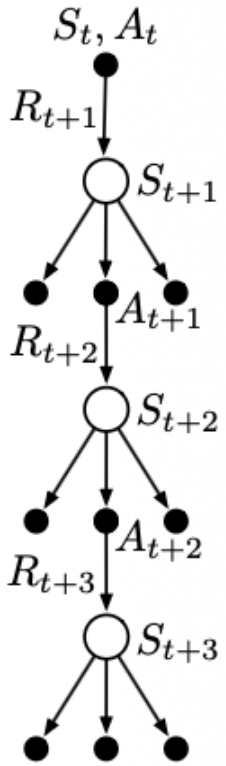
**One-step tree backup target,** Expected SARSA 와 같음

$$G_{t:t+1} = R_{t+1} + \gamma \sum_{a} \pi(a|S_{t+1}) Q_t(S_{t+1}, a)$$

**Multi-step tree backup target.**

$$G_{t:t+n} = R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_t(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1:t+n}$$

* Importance sampling을 우회하여, 높은 variance 문제에서 자유롭다.




the 3-step tree-backup update

# TD(λ)

MC (full episode) 와 TD (one-step) 을 보간하고자 n-step을 도입하였다.

하지만 에피소드마다 step 수는 정해져 있지 않고

환경이나 학습 단계에 따라서도 적절한 n은 다를 수 있다.

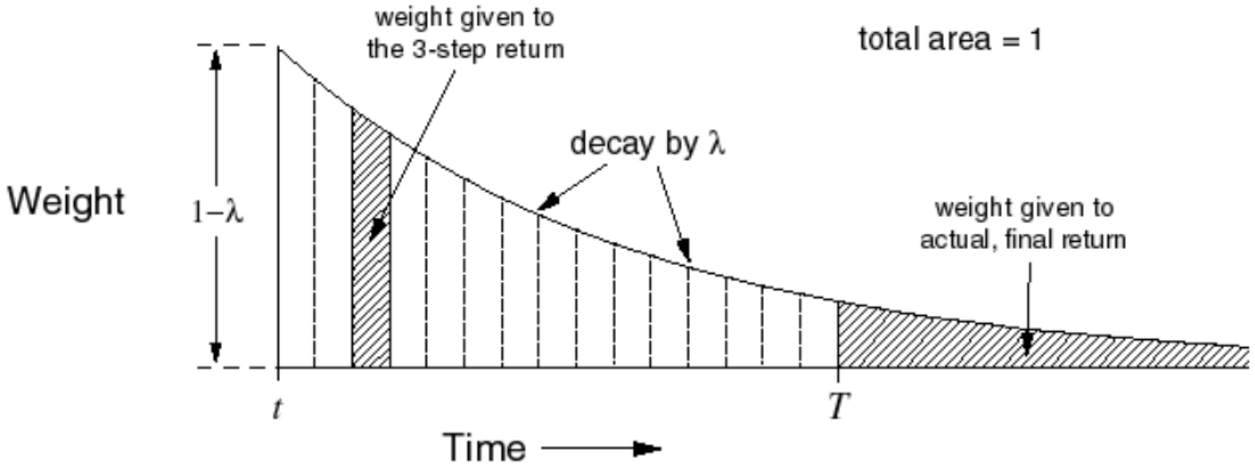
**고정된 n이 과연 최선일까?**

**→ 모든 n-step return 을 가중평균하자, TD(λ) : Forward view**

# TD(λ) – Forward View

**λ -return**

$$G_t^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$



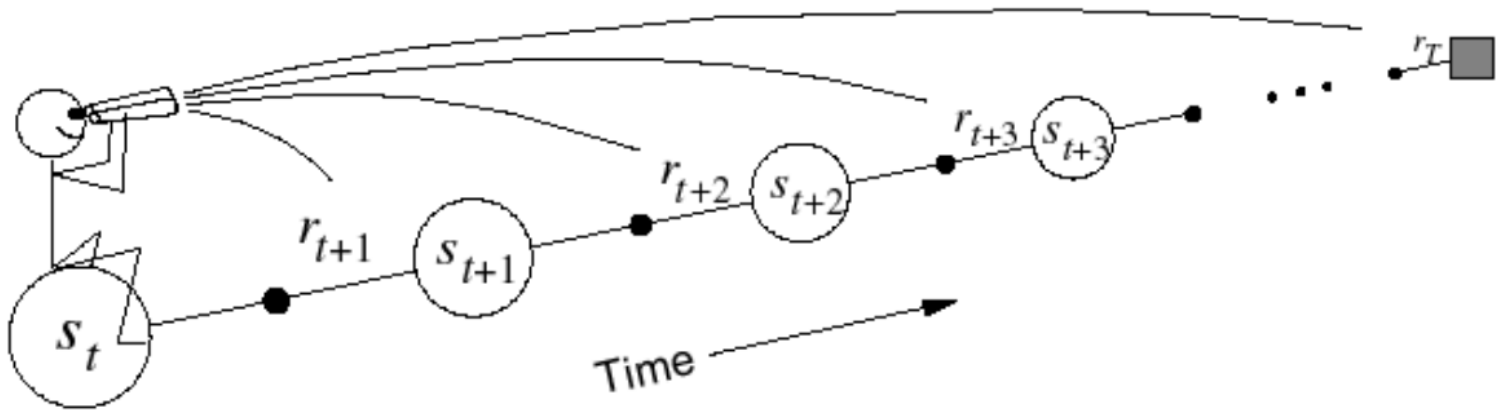

**Forward view of TD(λ)**

$$V(S_t) \leftarrow V(S_t) + \alpha\left(G_t^{\lambda} - V(S_t)\right)$$

**λ=0 → TD(0),**

**λ=1 → MC**

# TD(λ) – Forward View




**Forward View TD(λ)는 가치함수를 업데이트하기 위해 $G_t^\lambda$를 계산해야함.**

**그런데 $G_t^\lambda$ 계산을 위해서는 에피소드가 끝나야 함.**

**이러면 그냥 MC 쓰지 왜 복잡하게 계산량을 늘려가면서 TD(λ)를 쓰는거임?**

**→ Bootstrap return 과 실제 return이 섞여있기 때문에 Variance가 낮음, 그리고 …**

# Eligibility Trace

TD(0)을 먼저 생각해보자.

상태 $S_t$에서 TD Error $\delta_t$가 발생했을 때, TD(0)는 현재 state만 업데이트함

.

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t), \qquad V(S_t) \leftarrow V(S_t) + \alpha\delta_t$$

이건 보상에 대한 정보가 한 칸씩 전파됨을 의미함.

우리는 λ-return을 설계할 때 에피소드가 끝나는 시점까지의 TD Target을 가중평균했음

**→ 상태 $S_t$에서 TD Error $\delta_t$가 발생했을 때, 지나온 모든 state가 같이 업데이트되어야 함.**

# Eligibility Trace

**Eligibility Trace - 이 상태가 $S_t$에서 발생한 TD Error $\delta_t$를 받을 자격이 얼마나 있는가?**

최근 방문한 state에 더 높은 자격을 부여하자! → Recency heuristic

자주 방문한 state에 더 높은 자격을 부여하자! → Frequency heuristic

$$E_0(s) = 0, \qquad E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}(S_t = s)$$

# TD(λ) – Backward View

**Forward view가 이론적 배경이라면 Backward view는 메카니즘이다.**

모든 상태 s 에 대해 Eligibility Trace를 기록하고, Trace가 0이 아닌 s 에 대해 V를 업데이트.

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s)$$

이때 $\lambda = 0$이면, TD(0)와 동일하다.

$$E_t(s) = \mathbf{1}(S_t = s), \qquad V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

# TD(λ) – Backward View

$\lambda \neq 0$인 경우는 조금 복잡하다.

**V 업데이트 없이 한 에피소드 내의 모든 TD(λ) 업데이트를 Backward로 계산하는 경우,**

$$\sum_{t=1}^{T} \alpha \delta_t E_t(s) = \sum_{t=1}^{T} \alpha (G_t^{\lambda} - V(S_t))\, \mathbf{1}(S_t = s)$$

위의 식이 성립하고 TD(1), MC 업데이트 량과 동일하다 (Offline TD(λ))

**Online TD(λ)의 경우 매 업데이트에서 가치함수 V가 업데이트되고 해당 업데이트가 이후의 TD Error 계산에 영향을 미치기 때문에, Forward view 와 완전히 같지 않다**

*Sutton이 이 격차까지 보정한 Exact Online TD(λ)를 만들었다고 한다.

# TD(λ)

| Offline updates | $\lambda = 0$ | $\lambda \in (0,1)$ | $\lambda = 1$ |
|---|---|---|---|
| Backward view | TD(0) | TD($\lambda$) | TD(1) |
| | $\parallel$ | $\parallel$ | $\parallel$ |
| Forward view | TD(0) | Forward TD($\lambda$) | MC |
| **Online updates** | $\lambda = 0$ | $\lambda \in (0,1)$ | $\lambda = 1$ |
| Backward view | TD(0) | TD($\lambda$) | TD(1) |
| | $\parallel$ | $\nparallel$ | $\nparallel$ |
| Forward view | TD(0) | Forward TD($\lambda$) | MC |
| | $\parallel$ | $\parallel$ | $\parallel$ |
| Exact Online | TD(0) | Exact Online TD($\lambda$) | Exact Online TD(1) |

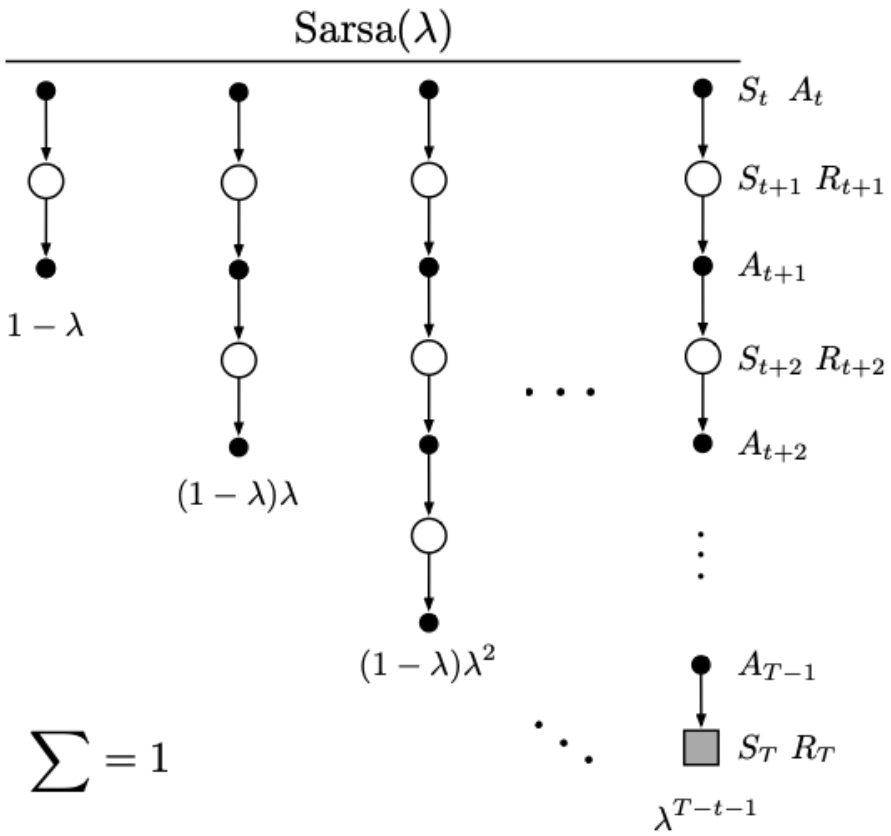
# SARSA(λ) – Forward View

$q^{\lambda}$ **-return**

$$q_t^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1}\, q_t^{(n)}$$

**Forward view of SARSA(λ)**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(q_t^{\lambda} - Q(S_t, A_t)\right)$$



# SARSA(λ) – Backward View

**Eligibility Trace (State-Action)**

$$E_0(s,a) = 0, \qquad E_t(s,a) = \gamma\lambda E_{t-1}(s,a) + \mathbf{1}(S_t = s, A_t = a)$$

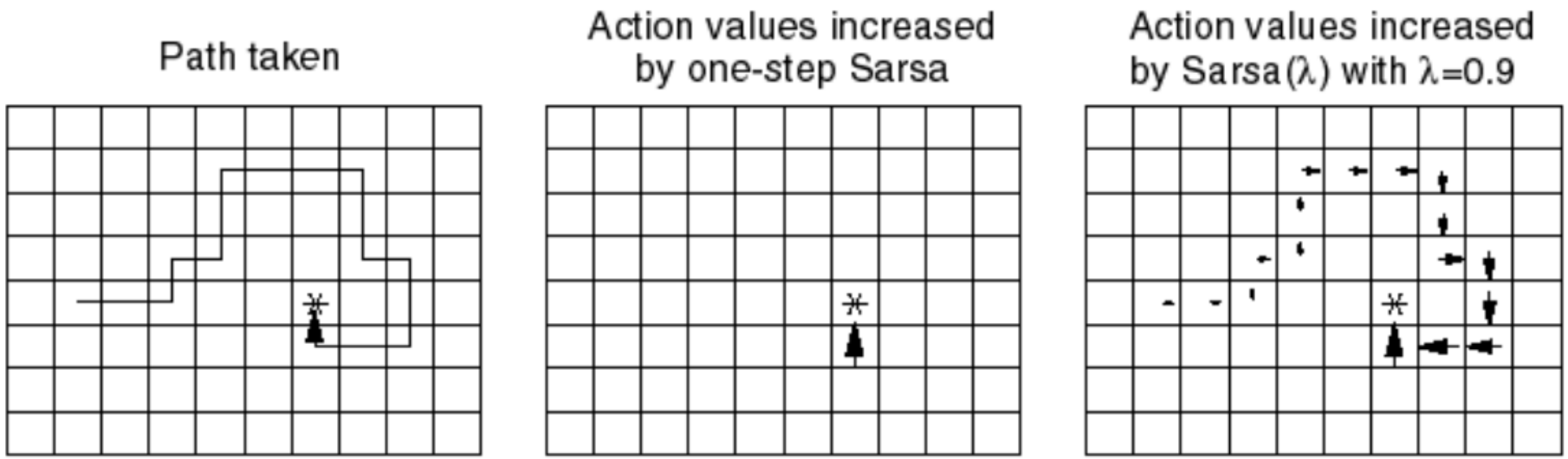
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left(q_t^{\lambda} - Q(S_t, A_t)\right)$$

상태 $S_t$, 행동 $A_t$ 에서 TD Error $\delta_t$가 발생했을 때,

.

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t), \qquad Q(s,a) \leftarrow Q(s,a) + \alpha\delta_t E_t(s,a)$$

# SARSA(λ)


Path taken
Action values increased by one-step Sarsa
Action values increased by Sarsa(λ) with λ=0.9

# Thank You